

Basile DUFRENE

Evan RIBOULEAU

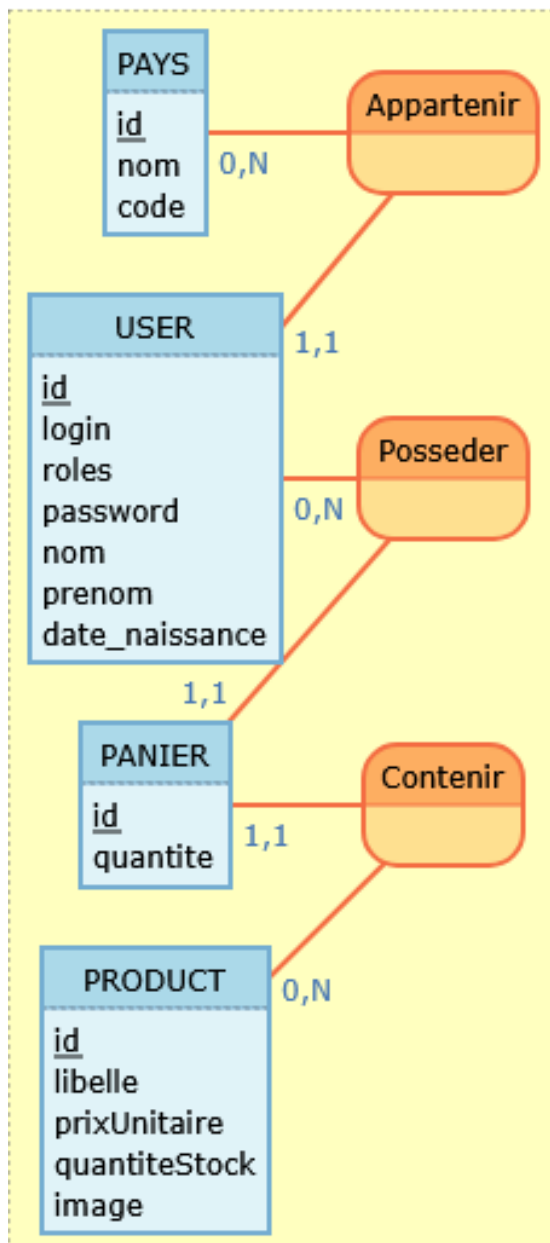
Rapport du projet_tws



Table des matières

Schéma de la base de données	-----p2
Organisation du code	-----p3/4
Remarques sur le Framework symfony	-----p5/6

Schéma de la base de données :



Organisation du code :

L'application repose sur le framework Symfony 7.3, respectant une architecture stricte qui sépare la logique métier, la gestion des données et l'affichage.

-Hiérarchie du projet :

Le code source est localisé dans le répertoire /src et les vues dans /templates.

- src/Entity/ : Contient les classes PHP qui définissent la structure de la base de données (User, Product, Panier, Pays). Chaque entité utilise des Attributs PHP 8 pour le mapping ORM (Doctrine).
- src/Controller/ : Point d'entrée des requêtes. Les contrôleurs récupèrent les données via les Repositories et les transmettent aux vues.
- src/Repository/ : Classes générées par Doctrine permettant d'effectuer des requêtes SQL complexes de manière fluide en PHP.
- src/DataFixtures/ : Script de peuplement de la base (AppFixtures.php) permettant de générer automatiquement les comptes de test et le catalogue de produits.
- templates/ : Utilise le moteur Twig. L'organisation est découpée par dossiers de contrôleurs (admin/, sadmin/, produit/) pour une maintenance facilitée.

-Les classes :

Dossier src/Entity/ (Le Modèle)

- User.php : Gère les informations des utilisateurs (login, nom, prénom, mot de passe), leurs rôles (Client, Admin, S-Admin) et la relation avec leur pays et leur panier.
- Product.php : Définit les produits (blocs Minecraft) avec leur libellé, prix unitaire, quantité en stock et le nom du fichier image associé.
- Pays.php : Entité de référence contenant le nom et le code des pays disponibles pour les utilisateurs.
- Panier.php : Entité de liaison (table de fait) stockant la relation entre un utilisateur et un produit, ainsi que la quantité commandée.

Dossier src/Controller/ (La Logique)

- DefaultController.php : Gère la route racine (/), définit le statut de l'utilisateur pour l'affichage et redirige vers la page d'accueil.
- ProductController.php : Contrôle l'affichage du catalogue et traite l'ajout ou la mise à jour des produits dans le panier (gestion du stock m+n).
- PanierController.php : Permet de visualiser le contenu du panier, de modifier les quantités finales ou de supprimer des lignes avant validation.
- SadminController.php : Espace sécurisé. Gère exclusivement la liste des administrateurs, leur création et la révocation de leurs droits.

- AdminController.php : Permet aux administrateurs de gérer les clients et d'ajouter de nouveaux produits (blocs) au catalogue, en plus des fonctionnalités users qu'ils disposent.
- RegistrationController.php : Gère le formulaire d'inscription public pour les nouveaux clients.
- SecurityController.php : Gère les mécanismes de connexion (login) et de déconnexion (logout).

Dossier src/Form/ (Les Formulaires)

- RegistrationFormType.php : Structure du formulaire d'inscription (utilisé pour les clients et réutilisé par le S-Admin pour les admins).
- ProductType.php : Structure du formulaire permettant à l'administrateur d'ajouter ou modifier un bloc Minecraft.

Dossier src/DataFixtures/ (Données de test)

- AppFixtures.php : Script automatisé générant les données initiales : les pays, les utilisateurs de test (rita, mathieu, sadmin) et le catalogue de blocs.

Remarques sur le Framework symfony:

Cette section détaille les fonctionnalités avancées de Symfony mises en œuvre pour répondre aux exigences spécifiques du cahier des charges.

-Gestion de la Sécurité :

Le point central du projet a été la mise en place d'une politique de sécurité différenciée pour les quatre types d'utilisateurs.

- Isolation du Super-Administrateur : Contrairement à une hiérarchie classique où le "Super" a tous les droits, nous avons configuré le menu et les contrôleurs pour que le Super-Admin soit strictement cantonné à la gestion des administrateurs. Il n'a pas accès à la boutique (produits/panier), respectant ainsi la consigne de séparation des tâches.
- Contrôle d'accès (access_control) : Dans le fichier security.yaml, nous avons verrouillé les routes /produits et /panier pour les rendre invisibles aux visiteurs anonymes, les redirigeant automatiquement vers le formulaire de connexion.

- Hashage des mots de passe : Pour chaque création de compte (client ou admin), nous utilisons UserPasswordHasherInterface pour garantir que les mots de passe ne transitent jamais en clair dans la base SQLite.

-Logique métier du Panier (Règle m+n) :

La gestion du panier a nécessité une logique Twig et Controller particulière pour respecter la règle de quantité :

- Calcul dynamique : Dans le site_layout.html.twig, le nombre d'articles affiché dans le menu est recalculé en temps réel en bouclant sur la collection app.user.paniers.
- Menu déroulant adaptatif : Sur la page produit, le formulaire d'ajout propose un menu allant de 0 à m (quantité en stock) + n (quantité déjà présente dans le panier). Cela permet à l'utilisateur de réajuster sa commande globalement sans jamais dépasser le stock physique disponible.

-Intégration des médias (Assets) :

Pour le catalogue de blocs Minecraft, nous avons optimisé la gestion des images :

- Contrainte de poids : Toutes les images utilisées pèsent moins de 50 Ko.
- Rendu visuel : Utilisation de la propriété CSS image-rendering: pixelated; dans Twig. Cette technique permet de conserver la netteté des pixels des blocs Minecraft lors du redimensionnement, évitant l'effet de flou natif des navigateurs.
- Persistance : Seul le nom du fichier (ex: herbe.png) est stocké en base de données, le fichier physique étant servi via le composant Asset de Symfony depuis le dossier /public.

